

중 간 발 표 — 2026년 4월

GPU 클러스터 추론 워크로드를 위한 스케줄링 및 배치 정책 공동 최적화

추론 지연시간 SLO 준수와 GPU 활용률 극대화를 위한 통합 정책 설계

윤영준 · 전현성 · 박준열 · 부광민

젠슨항팀 · 컴퓨터종합설계



EXECUTIVE SUMMARY

기존 GPU 스케줄러는 추론 SLO를 체계적으로 위반하며, 비대칭 스코어링이 이를 해결한다

학습(training)에 최적화된 기존 정책을 추론(inference)에 그대로 적용하면, 응답 시간 보장(SLO)이 무너진다. SLO 마감일 기반의 비대칭 가중치로 긴급 작업을 즉시 처리한다.

문제

추론은 짧은 실행시간과 엄격한 SLO를 요구하나, FIFO/SRTF/LAS는 "언제까지 끝나야 하는가"를 전혀 고려하지 않는다.

접근

SLO 마감까지의 여유(slack)를 비대칭 스코어링한다. 위반 임박 시 10× 가중치로 즉시 우선 실행하고, 배치 정책과 공동 최적화한다.

검증

Blox 시뮬레이터 위에 4종 스케줄러를 구현하고, 실제 Philly Trace(117K jobs) 고부하 환경에서 SloScoring이 FIFO 대비 JCT 62% 감소를 달성했다.

4

스케줄링 정책 구현

117K

실제 Philly Trace 작업

10×

SLO 위반 가중치 (α/β)

62%↓

JCT 감소 (vs FIFO)

PROBLEM DEFINITION

추론과 학습은 근본적으로 다른 워크로드이다

같은 GPU 클러스터 위에서 돌아가지만, 요구하는 것은 정반대다.

EXHIBIT 1

학습 vs 추론 워크로드 비교

특성	학습 (TRAINING)	추론 (INFERENCE)	시사점
실행 시간	수 시간 ~ 수 일	수 초 ~ 수 분	스케줄링 빈도 ↑↑
도착 패턴	소수의 대규모 작업	다수의 소규모 작업	큐 경쟁 심화
핵심 지표	총 완료 시간 (JCT)	응답 시간 보장 (SLO)	마감 시한 개념 필요
실패 비용	재학습 (체크포인트 복구)	사용자 이탈, SLA 위반	즉각 비즈니스 영향
선점 가능성	가능 (체크포인트 저장)	불가 (응답 소실)	선점 기반 스케줄링 불가

SO WHAT

학습용으로 설계된 FIFO, SRTF, LAS는 "이 작업이 언제까지 끝나야 하는가"를 전혀 고려하지 않는다. 추론 워크로드에는 SLO 마감 시한을 직접 반영하는 새로운 스케줄링 기법이 필요하다.

기존 세 정책 모두 SLO 개념이 부재하다

각 정책은 고유한 장점이 있으나, 추론의 핵심 요구인 "마감 준수"를 충족하지 못한다.

FIFO

First-In First-Out

먼저 제출된 작업을 먼저 실행한다. 구현이 간단하고 공정하다.

A(10hr) → B(1min) → C(30s)
B, C는 **10시간 대기**

Head-of-Line Blocking

SRTF

Shortest Remaining Time First

남은 시간이 가장 짧은 작업부터 실행. 이론적으로 평균 JCT 최적해.

실제 환경에서는 남은 시간을
미리 알 수 없으며,
긴 작업이 무한히 밀린다

Oracle Baseline

기아 문제

LAS

Least Attained Service

가장 적게 서비스받은 작업 우선. 실행 시간을 몰라도 적용 가능.

짧은 작업이 자연히 우선 처리
하지만 **SLO 마감 개념 없음**

SLO 무관

GAP

세 정책 모두 **작업의 긴급도(urgency)**를 판단하지 못한다. 추론 환경에서는 "아직 여유 있는 작업"과 "곧 마감인 작업"을 구분하는 메커니즘이 필수적이다.

기존 스케줄러는 추론 SLO와 배치를 동시에 최적화하지 못한다

학습 중심 스케줄러는 SLO 자체가 부재하고, 추론 서빙 시스템은 클러스터 수준 배치 공동 최적화가 없다.

EXHIBIT 2

관련 연구 비교 분석

논문	스케줄링	배치	대상	핵심 한계
학습 중심 스케줄러				
Tiresias NSDI'19	LAS + 2D 우선순위	Consolidated	Training	SLO 개념 부재
Pollux OSDI'21	Goodput 적응형	배치 크기 공동 조정	Training	SLO 개념 부재
Gavel OSDI'20	공정성 기반	Heterogeneity-aware	Training	추론 특화 아님
추론 서빙 시스템				
Clockwork OSDI'20	예측 기반 SLO	단일 모델	Inference	클러스터 배치 미지원
Shepherd OSDI'23	SLO-aware 서빙	모델 레벨	Inference	배치 공동 최적화 없음
프레임워크				
Blox EuroSys'24	모듈러 프레임워크	교체 가능	Training	정책 미제안, 추론 미검증
본 연구 Ours	비대칭 SLO 스코어링	부하 인식형	Inference	단일 클러스터 가정

OUR CONTRIBUTION

학습 중심 스케줄러에는 SLO가 부재하고, 추론 서빙 시스템에는 클러스터 수준 배치 최적화가 없다. 본 연구는 **SLO 마감일을 스케줄링에 직접 반영하면서 배치 정책과 공동 최적화**하는 최초의 추론 특화 연구다.

위반 임박 시 급격히, 여유 시 완만하게 — 비대칭이 핵심이다

SLO 마감까지의 여유(slack)를 비대칭 함수로 스코어링하여, 긴급 작업을 즉시 처리한다.

CORE ALGORITHM

비대칭 스코어링 함수

Slack이란? 작업이 SLO 마감까지 남은 여유 시간.

양수 = 여유 있음, **음수 = SLO 위반 임박**

† SLO deadline = 예상 완료 시간 × 2.0

1. Slack 계산

$$\text{slack} = \text{deadline} - (\text{now} + \text{remaining})$$

2. 비대칭 스코어링

$$\text{slack} < 0 \quad \text{score} = -10.0 \times |\text{slack}|$$

$$\text{slack} \geq 0 \quad \text{score} = 1.0 \times \text{slack}$$

3. 실행 순서

score가 **낮을수록** 먼저 실행

$\alpha = 10.0$ 위반 페널티 (급경사)

$\beta = 1.0$ 여유 보상 (완경사)

$\alpha \gg \beta$ 이 비대칭이 SLO 보호의 핵심

EXHIBIT 3

스코어링 함수 시각화



■ 위험 구간: 10배 가중치로 즉시 우선 실행 ■ 안전 구간: 완만하게 대기

SLO 스코어링과 부하 인식 배치가 파이프라인으로 연동된다

스케줄러가 SLO 기반 우선순위를 결정하면, 배치 정책이 GPU 부하를 고려하여 할당한다.



Blox 위에 커스텀 스케줄러를 구현하고 베이 스라인 3종을 재현했다

PROJECT STRUCTURE

```
# 시뮬레이션 실행
simulator_simple.py
run_scheduler.py          # [신규]

# 스케줄링 정책
schedulers/
  fifo.py
  las.py
  srtf.py
  slo_scoring.py          # [신규]

# 실험 도구
generate_synthetic_trace.py # [신규]
plot_results.py            # [신규]
```

CORE IMPLEMENTATION

SloScoring 핵심 코드

```
class SloScoring(SchedulingPolicy):
    alpha = 10.0 # SLO 위반 가중치
    beta  = 1.0  # 여유 가중치

    def schedule(self, job_dict, ...):
        for job in job_dict.values():
            deadline = job["submit_time"] + 2.0 *
job["duration"]
            slack = deadline - (now + job["remaining"])
            if slack < 0:
                job["score"] = -self.alpha * abs(slack)
            else:
                job["score"] = self.beta * slack
        return sorted(jobs, key=lambda j: j["score"])
```

ENVIRONMENT

시뮬레이터	Blox (EuroSys'24)
트레이스	MS Philly Trace (117K jobs)
Python	3.14
부하	8 jobs/hour

고부하 환경에서 SloScoring이 FIFO 대비 JCT 62% 감소, Oracle(SRTF) 수준 달성

EXHIBIT 5

Scheduling Policy Comparison (Real Philly Trace)

Load = 8 jobs/hr | 32 nodes × 4 GPUs | Microsoft Philly Trace (117K jobs) | 100 tracked jobs

정책	AVG JCT (S)	FIFO 대비	AVG 응답성 (S)
FIFO	147,361	기준	94,709
LAS	179,238	+21.6%	441
SRTF	55,255	-62.5%	1,519
SloScoring	55,486	-62.3%	468

정책	MEDIAN JCT	P95 JCT	P99 JCT
FIFO	109,726	310,583	529,336
LAS	17,343	217,283	4,615,936
SRTF	17,343	217,283	468,136
SloScoring	17,343	217,283	431,536

핵심 발견: SloScoring은 Oracle(SRTF)과 동등한 JCT를 달성하면서, 응답성은 SRTF(1,519s) 대비 3.2배 빠른 468초. P99에서도 SRTF보다 8% 낮은 tail latency.

ANALYSIS

정책 간 차이 해석

- 1. FIFO: Head-of-line blocking
긴 작업이 GPU를 점유하면 뒤의 짧은 작업이 94,709초(26시간) 대기. 응답성 최악.
- 2. LAS: Preemption 역설
짧은 작업은 즉시 시작(441s)하지만, 긴 작업의 반복 preemption으로 P99 JCT가 4,615,936초로 폭증.
- 3. SloScoring: 최적 균형
SRTF급 JCT + LAS급 응답성을 동시 달성. SLO 마감 기반 우선순위가 Oracle 수준 성능을 비Oracle 방식으로 구현.

SO WHAT

SloScoring은 작업 소요시간을 미리 알 필요 없이(non-Oracle) Oracle 수준의 JCT와 최고 수준의 응답성을 동시에 달성한다.

Blox 재현 환경을 구축하고 4건의 호환성 이슈를 해결했다

EXHIBIT 6

이슈 해결 내역

#	문 제	원 인	해 결	상 태
01	protobuf API 변경	v7.x에서 including_default_value_fields 이름 변경	always_print_fields_with_no_presence	해결
02	DataFrame.append() 삭제	pandas 3.x에서 deprecated 메서드 완전 제거	pd.concat() 으로 대체	해결
03	gRPC 스텝 미포함	Blox 레포에 컴파일된 Python 스텝 미배포	make grpc 수동 컴파일	해결
04	Philly Trace LFS 포인터	GitHub LFS pointer(135B)만 클론됨, 실제 데이터(1GB) 미포함	LFS media URL 직접 다운로드 (117K jobs 확보)	해결

IMPLICATION

이 작업 덕분에 Blox 시뮬레이터 실험이 **완전히 재현 가능**하다. 환경 구축 과정을 문서화하여, 향후 팀원 누구든 동일한 실험을 즉시 수행할 수 있는 기반을 확보했다.

실험 인프라 완비 + 실제 Philly Trace 실험 완료 — 최적화 단계 착수

STATUS TRACKER

● 주제 확정 및 문헌 조사	완료
● Blox 시뮬레이터 환경 구축 + 호환성 해결	완료
● 실제 Philly Trace 확보 (117K jobs)	완료
● 베이스라인 스케줄러 3종 실행 (FIFO/LAS/SRTF)	완료
● SloScoring 프로토타입 구현 + 고부하 실험 완료	완료
● 실험 결과: SloScoring JCT -62% (vs FIFO)	완료
● 실험 결과 시각화 파이프라인 (3종)	완료
● 부하 인식형 배치 정책 구현	8~9주차
● ML 기반 JCT 예측 프로토타이핑	10주차



전체 진행률 (공수 가중치 기준)

남은 과제

- 1. α/β 파라미터 민감도 분석
- 2. 부하 인식형 배치 정책 구현
- 3. ML 기반 JCT 예측 모델 통합
- 4. multi-GPU 작업 비중 확대 실험

ROADMAP

남은 7주: 고부하 실험 → ML 프로토타이핑 → 최종 검증

●	●	○	○	○
W1-6	W7 (NOW)	W8-9	W10-11	W12-14
환경 구축	중간발표	고부하 실험	ML 프로토타이핑	최종 발표
문헌조사, Blox 세팅	보고서 작성	α/β 파라미터 튜닝	JCT 예측 모델	성능 분석
베이스라인 재현	발표자료 제작	배치 정책 구현	파이프라인 통합	최종 보고서
프로토타입 구현	멘토 자문	contention 검증	파일럿 실험	코드 정리

EXHIBIT 7

팀원별 역할 배분

팀원	W 8-9	W 10-11	W 12-14
윤영준 Lead	SloScoring α/β 파라미터 튜닝	ML 파이프라인 프로토타이핑	최종 발표 총괄
전현성	Load-Balanced 배치 구현	ML 통합 지원 + 파일럿 실험	논문 정리
박준열	후보 ML 모델 벤치마크 보고서	실험 결과 시각화	최종 보고서
부광민	통합 테스트 + Trace 확보	성능 분석	대시보드 구축

KEY TAKEAWAY

SLO 마감일을 비대칭 가중치로 스케줄링에 직접 반영하면,
추론 워크로드의 **응답 시간 보장**과 **GPU 활용률**을 동시에 개선할 수 있다.

Q & A

젠슨황팀 · 컴퓨터종합설계